

PGR 210 - Natural Language Processing Part

By Huamin Ren

Huamin.ren@kristiania.no

Outline

- 1. From word counts to topics scores
 - 1.1 Topic vectors
 - 1.2 Thought experiment
- 2. LDA classifier
- 3. Latent Semantic Analysis (LSA)

Outline

- **1. *From word counts to topics scores***
 - 1.1 Topic vectors
 - 1.2 Thought experiment
- 2. LDA classifier
- 3. Latent Semantic Analysis (LSA)

1.1 Important concepts

- A document: an entity containing a whole body of text data with optional headers and other metadata information.
- A corpus: usually consists of a collection of documents. These documents can be simple sentences or complete paragraphs of textual information.
- A tokenized corpus: each document is tokenized or broken down into tokens, which are usually words.

Topic model

- The idea of topic models: revolves around the process of extracting key themes or concepts from a corpus of documents
- Each topic can be represented as a bag or collection of words/terms from the document corpus. Together, these terms signify a specific topic, theme, or a concept and each topic can be easily distinguished from other topics by virtue of the semantic meaning conveyed by these terms

- TF-IDF: estimate the importance of words in a chunk of text.

By using TF-IDF vectors and matrices, it can tell how important each word is to the overall meaning of a bit of text in a document collection.

Not good enough to represent a topic!

- Processing steps for a given document:
 - Normalization: such as stemming and lemmatization to create small collections of words with similar spellings
 - Processed these new tokens instead of the original words
 - Form a TF-IDF for each document
- However:
 - Lemmatization approach kept similarly spelled words together in your analysis, but not necessarily with similar meanings
 - Two chunks of text that talk about the same thing but use different words will not be “close” to each other in the lemmatized TF-IDF vector

So we need LSA (Latent semantic analysis)

- LSA is an algorithm to analyze your TF-IDF matrix (table of TF-IDF vectors) to gather up words into topics.

It works on bag-of-words vectors, too, but TF-IDF vectors give slightly better results.

- The terms “topic,” “semantic,” and “meaning” have similar meaning and are often used interchangeably when talking about NLP.

Topic vectors

- When doing math on TF-IDF vectors, such as addition and subtraction, these sums and differences only tell you about the frequency of word uses in the documents whose vectors you combined or differenced, but doesn't tell much about the meaning behind those words
- Need a way to extract some additional information, meaning, from word statistics: word-topic vectors and document-topic vectors.

1.2 A thought experiment

- Imaging processing some sentences about pets in Central Park in New York City (NYC)
 - Create three topics: one about pets, one about animals, and another about cities – ‘petness’, ‘animalness’, and ‘cityness’.
- ‘petness’ : score higher in words as ‘cat’ and ‘dog’, but lowest on ‘lion’.

```

topic['petness'] = (.3 * tfidf['cat'] +\
                  .3 * tfidf['dog'] +\
                  0 * tfidf['apple'] +\
                  0 * tfidf['lion'] -\
                  .2 * tfidf['NYC'] +\
                  .2 * tfidf['love'])

topic['animalness'] = (.1 * tfidf['cat'] +\
                     .1 * tfidf['dog'] -\
                     .1 * tfidf['apple'] +\
                     .5 * tfidf['lion'] +\
                     .1 * tfidf['NYC'] -\
                     .1 * tfidf['love'])

topic['cityness'] = ( 0 * tfidf['cat'] -\
                    .1 * tfidf['dog'] +\
                    .2 * tfidf['apple'] -\
                    .1 * tfidf['lion'] +\
                    .5 * tfidf['NYC'] +\
                    .1 * tfidf['love'])

```

```

word_vector = {}
word_vector['cat'] = .3*topic['petness'] +\
                    .1*topic['animalness'] +\
                    0*topic['cityness']
word_vector['dog'] = .3*topic['petness'] +\
                    .1*topic['animalness'] -\
                    .1*topic['cityness']
word_vector['apple'] = 0*topic['petness'] -\
                    .1*topic['animalness'] +\
                    .2*topic['cityness']
word_vector['lion'] = 0*topic['petness'] +\
                    .5*topic['animalness'] -\
                    .1*topic['cityness']

word_vector['NYC'] = -.2*topic['petness'] +\
                    .1*topic['animalness'] +\
                    .5*topic['cityness']
word_vector['love'] = .2*topic['petness'] -\
                    .1*topic['animalness'] +\
                    .1*topic['cityness']

```

1.2 A thought experiment

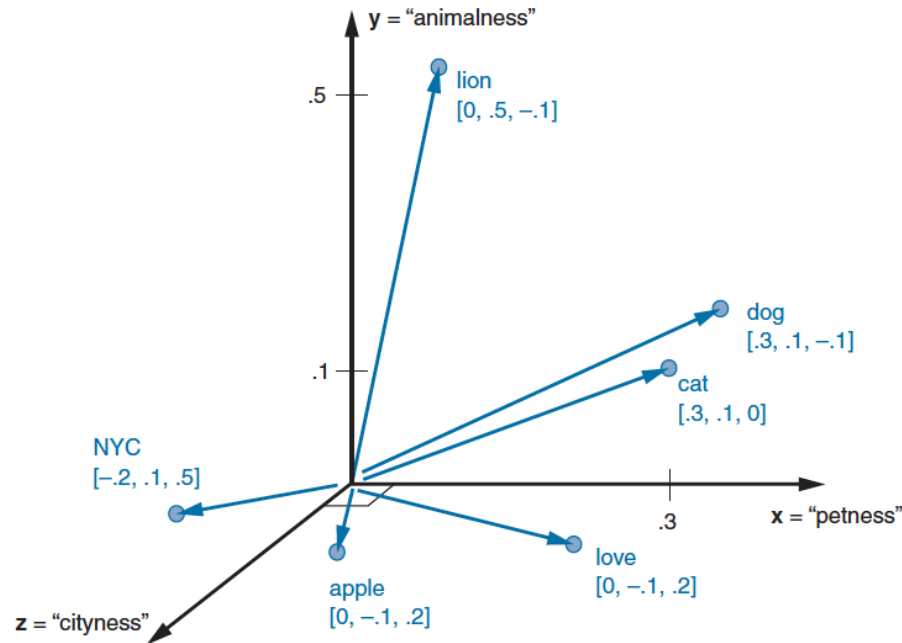


Figure 4.1 3D vectors for a thought experiment about six words about pets and NYC

4. Topic models

- 1. From word counts to topics scores
 - 1.1 Topic vectors
 - 1.2 Thought experiment
- **2. *LDA classifier***
- 3. Latent Semantic Analysis (LSA)

2. LDA classifier -Linear discriminant analysis

- LDA breaks down a document into only one topic.
- A simply LDA implementation:
 - Compute the average position (centroid) of all the TF-IDF vectors within the class (such as spam SMS messages).
 - Compute the average position (centroid) of all the TF-IDF vectors not in the class (such as nonspam SMS messages).
 - Compute the vector difference between the centroids (the line that connects them).

Outline

- 1. From word counts to topics scores
 - 1.1 Topic vectors
 - 1.2 Thought experiment
- 2. LDA classifier
- **3. *Latent Semantic Analysis (LSA)***

3. Latent semantic analysis (LSA)

- LSA is an algorithm to analyze your TF-IDF matrix (table of TF-IDF vectors) to gather up words into topics.
- The terms “topic,” “semantic,” and “meaning” have similar meaning and are often used interchangeably when talking about NLP.

3. Latent semantic analysis (LSA)

- Latent semantic analysis is based on the oldest and most commonly used technique for dimension reduction, singular value decomposition.
- A similar but simpler algorithm: Linear discriminant analysis (LDA)
- A similar but can break down documents into many topics: Latent Dirichlet allocation (LDA)